4: Working with Pods

Pods are fundamental to running apps on Kubernetes. As such, this chapter goes into quite a bit of detail.

The chapter is divided into two main parts:

- Theory
- Hands-on

Before getting started, it's difficult to talk about Pods without making reference to workload controllers such as Deployments, DaemonSets, and StatefulSets. However, this is the start of the book and we haven't covered any of those yet. So, we'll take a quick minute here to set the scene so that when they come up in the chapter, you'll have a basic idea of what they are.

You'll almost always deploy Pods via higher-level workload controllers – from now on, we'll just call them *controllers*.

Controllers gift Pods with super-powers such as self-healing, scaling, updates and rollbacks. You'll see this later, but every controller has a *PodTemplate* defining the Pods it deploys and manages. So, even though you'll rarely interact directly with Pods, it's absolutely vital you have a solid understanding of them.

For these reasons, we'll cover quite a lot of Pod detail. It won't be wasted time and will be very useful as you progress to controllers and other more advanced objects. You also need a deep understanding of Pods if you want to master Kubernetes.

With that out of the way, let's crack on.

Pod theory

The *atomic unit of scheduling* on Kubernetes is the Pod. This is just a fancy way of saying apps deployed to Kubernetes always run inside Pods.

Some quick examples… If you deploy an app, you deploy it in a Pod. If you terminate an app, you terminate its Pod. If you scale an app up or down, you add or remove Pods.

Why Pods

The process of building and running an app on Kubernetes is roughly as follows:

1. Write your app/code
2. Package it as a container image
3. Wrap the container image in a Pod
4. Run it on Kubernetes

This begs the question, *why not just run the container on Kubernetes?*

The short answer is *you just can't.* Kubernetes doesn't allow containers to run directly on a cluster, they always have to be wrapped in a Pod.

Broadly speaking, there are three main reasons for Pods.

1. Pods augment containers
2. Pods assist in scheduling
3. Pods enable resource sharing

Pods augment containers

On the augmentation front, Pods augment containers in all of the following ways.

- Labels and annotations
- Restart policies
- Probes (startup probes, readiness probes, liveness probes, and potentially more)
- Affinity and anti-affinity rules
- Termination control
- Security policies
- Resource requests and limits

Run a `kubectl explain pods` command to list all possible Pod attributes. Beware, the command returns over 1,000 lines and the following output has been trimmed.

```
$ kubectl explain pods --recursive
KIND:     Pod
VERSION:  v1

DESCRIPTION:
     Pod is a collection of containers that can run on a host. This resource is
     created by clients and scheduled onto hosts.

FIELDS:
   apiVersion             <string>
   kind                   <string>
   metadata               <Object>
      annotations         <map[string]string>
      labels             <map[string]string>
      name               <string>
      namespace           <string>
<Snip>
```

It's a useful command for finding which properties any object supports. It also shows the format of properties (string, map, object etc.).

Even more useful, is the ability to drill into specific attributes. The following command drills into the restart policy attribute of a Pod object.

```
$ kubectl explain pod.spec.restartPolicy
KIND:     Pod
VERSION:  v1
FIELD:    restartPolicy <string>
DESCRIPTION:
     Restart policy for all containers within the pod. One of Always, OnFailure, Never.
     Default to Always.
     More info: https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/...
```

You'll see a lot of Pod features as you progress through the book. However, it's worth a quick introduction to some of them right now.

Labels let you group Pods and associate them with other objects in powerful ways. *Annotations* let you add experimental features and integrations with 3rd-party tools and services. *Probes* let you test the health and status of Pods, enabling advanced scheduling, updates, and more. *Affinity* and *anti-affinity* rules give you control over where Pods run. *Termination control* lets you to gracefully terminate Pods and the applications they run. *Security policies* let you enforce security features. *Resource requests and limits* let you specify minimum and maximum values for things like CPU, memory and disk IO.

Despite bringing so many features to the party, Pods are super-lightweight and add very little overhead.

Figure 4.1 shows a Pod as a wrapper around one or more containers.

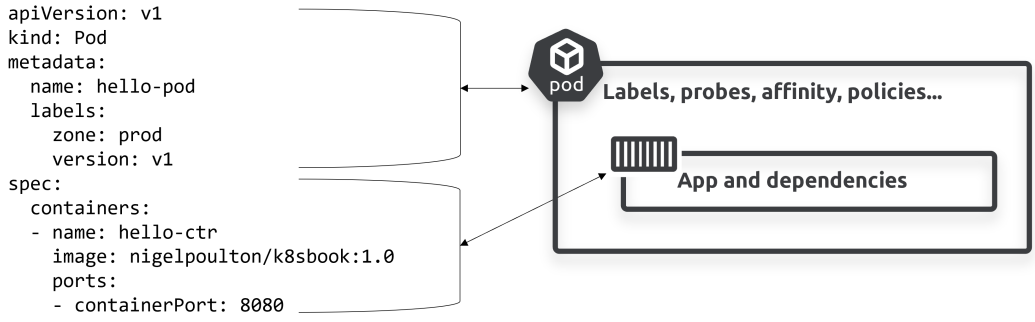

Figure 4.1

Pods assist in scheduling

On the scheduling front, every container in a Pod is guaranteed to be scheduled to the same cluster node. This in turn guarantees they'll be in the same region and zone in your cloud or datacenter. We call this *co-scheduling* and *co-locating*.

Labels, affinity and anti-affinity rules, and resource requests and limits give you fine-grained control over which nodes Pods can run on.

> **Note:** Remember that nodes are host servers that Pods run on. They can be physical servers, virtual machines, or cloud instances.

Pods enable resource sharing

On the sharing of resources front, Pods provide a *shared execution environment* for one or more containers. This shared execution environment includes things such as.

- Shared filesystem
- Shared network stack (IP address and ports…)
- Shared memory
- Shared volumes

You'll see it later, but every container in a Pod shares the Pod's execution environment. So, if a Pod has two containers, both containers share the Pod's IP address and can access any of the Pod's volumes to share data.

We'll cover all of these features throughout the book, but hopefully you understand some of the advantages of using Pods.

Static Pods vs controllers

There are two ways to deploy Pods.

1. Directly via a Pod manifest
2. Indirectly via a controller

Pods deployed directly from a Pod manifest are called *static Pods* and have no super-powers such as self-healing, scaling, or rolling updates. This is because they're only monitored and managed by the local kubelet process which is limited to attempting container and Pod restarts on the local node. If the node they're running on fails, there's no control-plane process watching and capable of starting a new one on a different node.

Pods deployed via controllers have all the benefits of being monitored and managed by a highly-available controller running on the control-plane. The local kubelet on the node they're running on can still attempt local restarts, but if restart attempts fail, or the node itself fails, the observing controller can start a replacement Pod on a different node.

Just to be clear, it's vital to understand that Pods as *mortal*. When they die, they're gone. There's no fixing them and bringing them back from the dead. This firmly places them in the *cattle* category in the *pets vs cattle* paradigm. Pods are *cattle*, and when they die, they get replaced by another. There's no tears and no funeral. The old one is gone, and a shiny new one – with the same config, but a different IP address and UID – magically appears and takes its place.

This is why applications should always store *state* and *data* outside the Pod. It's also why you shouldn't rely on individual Pods – they're ephemeral, here today, gone tomorrow…

In the real world, you'll almost always deploy and manage Pods via controllers.

Single-container and multi-container Pods

Pods can run one or more containers. The single-container model is the simplest, but multi-container Pods are important in real-world production environments and vital for service meshes. You'll learn more about multi-container Pods later in the chapter.

Deploying Pods

The process of deploying a Pod to Kubernetes is as follows.

1. Define it in a YAML *manifest file*
2. Post the YAML to the API server
3. The API server authenticates and authorizes the request
4. The configuration (YAML) is validated
5. The scheduler deploys the Pod to a healthy node with enough available resources
6. The local kubelet monitors it

If the Pod is deployed via a controller, the configuration will be added to the cluster store as part of overall desired state and a controller will monitor it.

Let's dig a bit deeper.

The anatomy of a Pod

At the highest level, a Pod is an execution environment shared by one or more containers. *Shared execution environment* means the Pod has a set of resources that are shared by every container it runs. These resources include IP address, ports, hostname, sockets, memory, volumes, and more…

It can be useful to think of Pods as shared environments, and containers as application processes.

If you're using Docker or containerd as your container runtime, a Pod is actually a special type of container called a "pause container". You heard that right, a Pod is just a fancy name for a special container. This means containers running inside of Pods are really containers running inside of containers. For more information, watch "Inception" by Christopher Nolan, starring Leonardo DiCaprio ;-)

Seriously though, a Pod is a collection of resources that containers running inside of it inherit and share. These resources are actually Linux kernel namespaces, and include the following:

- **net namespace**: IP address, port range, routing table…
- **pid namespace**: isolated process tree
- **mnt namespace**: filesystems and volumes…
- **UTS namespace**: Hostname
- **IPC namespace**: Unix domain sockets and shared memory

Let's look at how the Pod shared execution environment model affects networking.

Pods and shared networking

Each Pod creates its own network *namespace*. This means a Pod has its own IP address, a single range of TCP and UDP ports, and a single routing table. If it's a single-container Pod, the container has full access to the IP, port range and routing table. If it's a multi-container Pod, all containers share the IP, port range and routing table.

Figure 4.2 shows two Pods, each with its own IP. Even though one of them is a multi-container Pod, it still only gets a single IP.

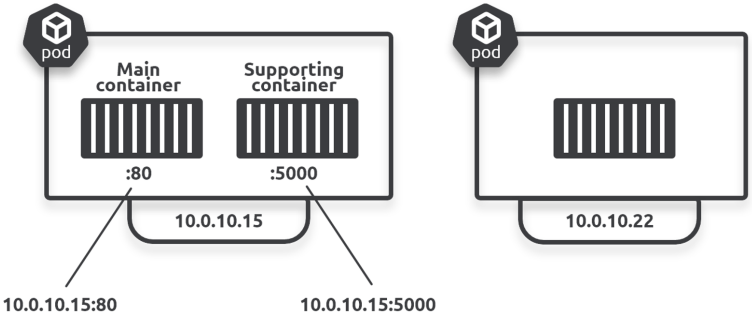

Figure 4.2

In Figure 4.2, external access to the containers in the Pod on the left is achieved via the IP address of the Pod coupled with the port of the container you're trying to reach. For example, `10.0.10.15:80` will get you to the main application container, but `10.0.10.15.5000` will get you to the supporting container.

Container-to-container communication within the same Pod happens via the Pod's `localhost` adapter and a port number. For example, the main container in Figure 4.2 can reach the supporting container on `localhost:5000`.

The pod network

On the topic of networking, every Pod gets its own unique IP addresses that's fully routable on an internal Kubernetes network called the *pod network*. The *pod network* is flat, meaning every Pod can talk directly to every other Pod without the need for complex port mappings. This is shown in Figure 4.3.

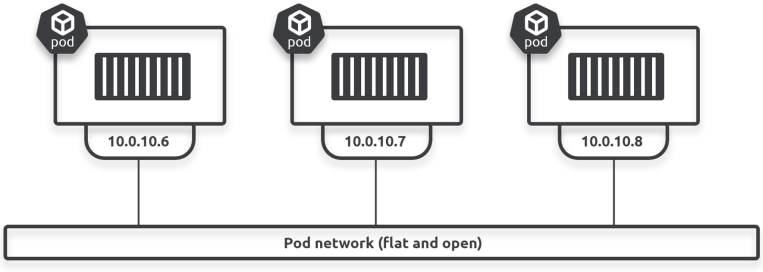

Figure 4.3 Inter-Pod communication

In a default out-of-the-box cluster, the Pod network is wide open from a security perspective. You should use Network Policies to lock down access.

Atomic deployment of Pods

Pod deployment is an *atomic operation*. This means it's all-or-nothing – deployment either succeeds or it doesn't. You'll never have a scenario where a partially deployed Pod is servicing requests. Only after all a Pod's resources are running and ready will it start servicing requests.

Pod lifecycle

The lifecycle of a typical Pod is something like this…

You define it in a declarative YAML object that you post to the API server and it enters the `pending` phase. It's then scheduled to a healthy node with enough resources and the local kubelet instructs the container runtime to pull all required images and start all containers. Once all containers are pulled and running, the Pod enters the *running* phase. If it's a short-lived Pod, as soon as all containers terminate successfully the Pod itself terminates and enters the *succeeded* state. If it's a long-lived Pod, it remains indefinitely in the `running` phase.

Shorted-lived and long-lived Pods

Pods can run all different types of applications. Some, such as web servers, are intended to be long-lived and should remain in the running phase indefinitely. If any containers in a long-lived Pod fail, the local kubelet may attempt to restart them.

We say the kubelet *"may"* attempt to restart them. This is based on the container's restart policy which is defined in the Pod config. Options include *Always*, *OnFailure*, and *Never*. *Always* is the default restart policy and appropriate for most long-lived Pods.

Other workload types, such as batch jobs, are designed to be short-lived and only run until a task completes. Once all containers in a short-lived Pod successfully terminate, the Pod terminates and its status is set to `successful`. Appropriate container restart policies for short-lived Pods will usually be *Never* or *OnFailure*.

Kubernetes has several controllers for different types of long-lived and short-lived workloads. *Deployments, StatefulSets,* and *DaemonSets* are examples of controllers designed for long-lived Pods. *Jobs* and *CronJobs* are examples designed for short-lived Pods.

Pod immutability

Pods are immutable objects. This means you can't modify them after they're deployed.

This can be quite a mindset change for some people, especially if you come from a background of deploying servers and regularly patching and updating them.

The immutable nature of Pods is a key aspect of cloud-native microservices design patterns and forces the following behaviors.

- When updates are needed, replace all old Pods with new ones that have the updates
- When failures occur, replace failed Pods with new identical ones

To be clear, you never actually update a running Pod, you always replace it with a new Pod containing the updates. You also never log onto failed Pods and attempt fixes, you build fixes into an updated Pod and replace failed ones with the updated one.

Pods and scaling

All Pods run a single application container instance, making them an ideal unit of scaling – if you need to scale the app, you add or remove Pods. This is call *horizontal scaling*.

You never scale an app by adding more of the same application containers to a Pod. Multi-container Pods are not a way to scale an app, they're only for co-scheduling and co-locating containers that need tight coupling.

Pod theory summary

1. Pods are the atomic unit of scheduling in Kubernetes
2. Single-container Pods are the simplest. However, multi-container Pods are ideal for co-locating tightly coupled workloads and are fundamental to service meshes
3. Pods get scheduled on nodes (host physical servers, VMs, cloud instances), and you can't schedule a single Pod to span multiple nodes
4. Pods are defined declaratively in manifest files you post to the API server
5. You almost always deploy Pods via higher-level controllers

Multi-container Pods

Multi-container Pods are a powerful pattern and heavily used in real-world environments.

At a very high-level, every container should have a single clearly defined responsibility. For example, an application that pulls content from a repository and serves it as a web page has two clear functions:

1. Pull the content
2. Serve the web page

In this example you should design two containers, one responsible for pulling the content and the other to serve the web page. We call this *separation of concerns* or *separation of responsibilities*.

This design approach keeps each container small and simple, encourages re-use, and makes troubleshooting simpler.

However, there are scenarios where it's a good idea to tightly couple two or more functions. Consider the same example app that pulls content and serves it via a web page. A simple design would have the "sync" container (the one pulling content updates) put content updates in a volume shared with the "web" container. For this to work, both containers need to run in the same Pod so they have access to the same shared volume in the Pod's shared execution environment.

Co-locating multiple containers in the same Pod allows containers to be designed with a single responsibility but work closely with others.

Kubernetes offers several well-defined multi-container Pod patterns.

- Sidecar pattern
- Adapter pattern
- Ambassador pattern
- Init pattern

Each one is an example of the *one-container-one-responsibility* model.

Sidecar multi-container Pods

The sidecar pattern is probably the most popular and most generic multi-container pattern. It has a main application container and a *sidecar* container. It's the job of the sidecar to augment or perform a secondary task for the main application container. The previous example of a main application web container, plus a helper pulling up-to-date content is a classic example of the sidecar pattern – the "sync" container pulling the content from the external repo is the *sidecar*.

An increasingly important user of the sidecar model is the service mesh. At a high level, service meshes inject sidecar containers into application Pods, and the sidecars do things like encrypt traffic and expose telemetry and metrics.

Adapter multi-container Pods

The *adapter pattern* is a specific variation of the generic sidecar pattern where the helper container takes non-standardized output from the main container and rejigs it into a format required by an external system.

A simple example is NGINX logs being sent to Prometheus. Out-of-the-box, Prometheus doesn't understand NGINX logs, so a common approach is to put an adapter sidecar into the NGINX Pod that converts NGINX logs into a format accepted by Prometheus.

Ambassador multi-container Pods

The *ambassador pattern* is another variation of the sidecar pattern. This time, the helper container brokers connectivity to an external system. For example, the main application container can just dump its output to a port the ambassador container is listening on and sit back while the *ambassador container* does the hard work of getting it to the external system.

It acts a lot like political ambassadors that interface with foreign nations on behalf of a government. In Kubernetes, ambassador containers interface with external systems on behalf of the main app container.

Init multi-container Pods

The init pattern is not a form of sidecar. It runs a special *init container* that's guaranteed to start and complete before your main app container. It's also guaranteed to only run once.

As the name suggests, it's job in life is to run tasks and *initialise* the environment for the main application container. For example, a main app container may need permissions setting, an external API to be up and accepting connections, or a remote repository cloning to a local volume. In cases like these, an init container can do that prep work and will only exit when the environment is ready for the main app container. The main app container will not start until the init container completes.

Hands-on with Pods

If you're following along, be sure to clone the book's GitHub repo and run all of the following commands from within the `pods` folder.

```
$ git clone https://github.com/nigelpoulton/TheK8sBook.git
Cloning into 'TheK8sBook'...
```

If you don't have `git`, or are uncomfortable using it, you can just visit the repo and copy the file contents into files with the same name on your local machine.

You can follow along with any of the Kubernetes clusters you saw how to build in the "Getting Kubernetes" chapter.

List any existing Pods to make it easier to identify the Pods you'll create in the following steps.

```
$ kubectl get pods
No resources found in default namespace.
```

Pod manifest files

You'll be using the following Pod manifest. It's available in the book's GitHub repo under the `pods` folder called `pod.yml`:

```
kind: Pod
apiVersion: v1
metadata:
  name: hello-pod
  labels:
    zone: prod
    version: v1
spec:
  containers:
  - name: hello-ctr
    image: nigelpoulton/k8sbook:1.0
    ports:
    - containerPort: 8080
```

Let's step through it and find out what it's describing.

Straight away you can see four top-level resources:

- `kind`
- `apiVersion`
- `metadata`
- `spec`

The `.kind` field tells Kubernetes the type of object being defined. This file is defining a Pod object.

`apiVersion` defines the schema version to use when creating the object. This file is defining a Pod object and telling Kubernetes to build it using the `v1` Pod schema.

The normal format for `apiVersion` is `<api-group>/<version>`. However, Pods are defined in a special API group called the *core* group which omits the *api-group* part. For example, StorageClass objects are defined in the `v1`

schema of the `storage.k8s.io` API group and are described in YAML files as `storage.k8s.io/v1`. However, Pods are in the *core* API group which omits the API group name, so we describe them in YAML files as just `v1`.

So far, you know you're deploying a Pod object as defined in `v1` of the *core API group*.

The `.metadata` section is where you attach things such as names, labels, annotations, and a Namespace. The name helps you identify the object in the cluster, and the labels let you create loose couplings with other objects. Annotations can help integrate with 3rd-party tools and services. We'll discuss Namespaces in a future chapter.

The `.metadata` section of this manifest is naming the Pod "hello-pod" and assigning it two labels. You'll use the labels in a future chapter to loosely couple it to a Service for stable networking. As it's not specifying a Namespace, it'll be deployed to the `default` Namespace. It's not good practice to use the default namespace in the real world, but it's fine for these examples.

The `.spec` section is where you define the containers the Pod will run. This is called the *Pod template*, and this example is defining a single-container Pod based on the `nigelpoulton/k8sbook:1.0` image. It's calling the container `hello-ctr` and exposing it on port `8080`.

If this was a multi-container Pod, you'd define additional containers in the `.spec` section.

Manifest files: Empathy as Code

Quick side-step.

Configuration files, like Kubernetes manifests, are excellent sources of documentation. As such, they have a few secondary benefits. A couple of these include:

- Speeding-up on-boarding of new team members
- Bridging the gap between developers and operations

For example, if you need a new team member to understand the basic functions and requirements of an application, get them to read its Kubernetes manifest files.

Also, if your operations teams complain that developers don't give accurate application requirements and documentation, make your developers use Kubernetes. This forces them to describe applications in Kubernetes manifests, which can then be used by operations to understand how the application works and what it requires from the environment. This is especially true in more advanced setups where you define things such as *resource requirements and limits* in manifest files.

These kinds of benefits were described as a form of *empathy as code* by Nirmal Mehta in his 2017 DockerCon talk entitled "A Strong Belief, Loosely Held: Bringing Empathy to IT".

Back on track…

Deploying Pods from a manifest file

You need to run the following commands from the `pods` folder where you cloned the book's GitHub repo to.

```
$ kubectl apply -f pod.yml
pod/hello-pod created
```

Although the Pod is showing as created, it might not be fully deployed and available yet. This is because it takes time to pull the image.

Run a `kubectl get pods` to check the status.

```
$ kubectl get pods
NAME        READY    STATUS             RESTARTS   AGE
hello-pod   0/1      ContainerCreating  0          9s
```

You can see the container is still creating – probably waiting while the container runtime pulls the image from Docker Hub.

This is a good time to mention that Kubernetes will pull (download) images from Docker Hub by default. To download from another registry, you need to specify the registry URL before the image name in the YAML file. The following image is in the Google Container Registry (k8s.gcr.io): `k8s.gcr.io/git-sync:v3.1.6`

You can add the `--watch` flag to the command to monitor it and see when the status changes to `Running`.

Congratulations. The Pod is running on a healthy node and is being monitored by the local `kubelet` process.

In future chapters, you'll see how to connect to the app running in the Pod.

Introspecting running Pods

The two main commands for checking the status of Pods are `kubectl get` and `kubectl describe`.

kubectl get

As good as `kubectl get pods` is, it's a bit light on detail. Not to worry though, there's plenty of options for deeper introspection.

First up, you can add a couple of flags that give you more information:

- `-o wide` gives a couple more columns but is still a single line of output
- `-o yaml` takes things to the next level, returning a full copy of the Pod from the cluster store.

The following command shows a snipped output from of a `kubectl get pods -o yaml` command. The output is broadly divided into two parts:

- desired state (`.spec`)
- observed state (`.status`)

```
$ kubectl get pods hello-pod -o yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      ...
  name: hello-pod
  namespace: default
spec:                          <<== Desired state is in this block
  containers:
  - image: nigelpoulton/k8sbook:1.0
    imagePullPolicy: IfNotPresent
```

```
    name: hello-ctr
    ports:
    ...
status:                         <<== Observed state is in this block
  conditions:
  - lastProbeTime: null
    lastTransitionTime: "2021-02-01T18:21:51Z"
    status: "True"
    type: Initialized
...
```

The full output contains a lot more than the 13-line YAML file you posted. So, where does this extra information come from?

Two main sources:

- Pod objects have a lot of properties. Anything not explicitly set in the YAML file is automatically expanded with default values.
- The `.status` section is the current observed state of the Pod.

kubectl describe

Another great command for introspection is `kubectl describe`. This provides a nicely formatted multi-line overview of an object. It even includes important object lifecycle events.

```
$ kubectl describe pods hello-pod
Name:         hello-pod
Namespace:    default
Start Time:   Mon, 01 Feb 2021 18:21:51 +0000
Labels:       version=v1
              zone=prod
Status:       Running
IP:           10.42.1.28
Containers:
  hello-ctr:
    Container ID:   containerd://4f66e48e...
    Image:          nigelpoulton/k8sbook:1.0
    Port:           8080/TCP
    ...
Conditions:
  Type              Status
  Initialized       True
  Ready             True
  ContainersReady   True
  ...
Events:
  Type    Reason     Age        Message
  ----    ------     ----     -------
  Normal  Scheduled  5m30s    Successfully assigned ...
  Normal  Pulling    5m30s    Pulling image "nigelpoulton/k8sbook:1.0"
```

```
  Normal  Pulled     5m8s     Successfully pulled image ...
  Normal  Created    5m8s     Created container hello-ctr
  Normal  Started    5m8s     Started container hello-ctr
```

The output has been snipped to fit the page.

kubectl logs

Another useful command for introspecting Pods is `kubectl logs`. Like other Pod-related commands, if you don't specify a container with the `--container` flag, it executes against the first container in the Pod. The format of the command is `kubectl logs <pod>`.

Each container in a multi-container Pod gets a name. The first Pod in the YAML object is the first container, and all commands will run against this unless you specify otherwise. If you need the command to run against another container, just add the `--container` flag and specify the container name.

The following YAML snippet is from a multi-container Pod called "multipod" that defines a container called `app` and another called `syncer`.

```
spec:
  containers:
  - name: app                   <<==== First container
    image: nginx
      ports:
        - containerPort: 8080
  - name: syncer                <<==== Second container
    image: k8s.gcr.io/git-sync:v3.1.6
    volumeMounts:
    - name: html
<Snip>
```

The following command runs against the `syncer` container.

```
$ kubectl logs multipod --container syncer
<Snip>
```

kubectl exec: running commands in Pods

Another way to introspect a running Pod is to log into it or execute commands in it. You can do both of these with `kubectl exec`. The following example shows how to execute a `ps aux` command in the first container in the `hello-pod` Pod.

```
$ kubectl exec hello-pod -- ps aux
PID   USER     TIME   COMMAND
  1   root     0:00   node ./app.js
 11   root     0:00   ps aux
```

The output shows the main application process (`app.js`) is running.

It's also possible to use `kubectl exec` to get shell access to a container running in a Pod. When you do this, your terminal prompt will change and you'll be able to execute commands from inside the Pod (as long as the command binaries are installed in it).

The following `kubectl exec` command will log-in to the first container in the `hello-pod` Pod. Once inside, install the `curl` utility and run a `curl` command to transfer data from the process listening on port `8080`.

> **Note:** This example is just for demonstration purposes. Installing tools inside running Pods and containers is an anti-pattern and should be avoided. If you need to make changes to a Pod, you should create a new Pod with the updates and replace old ones with the new one.

```
$ kubectl exec -it hello-pod -- sh

# apk add curl
<Snip>

# curl localhost:8080
<html><head><title>K8s rocks!</title><link rel="stylesheet" href="http://netdna....
```

The `-it` flags on the `kubectl exec` command make the session interactive and connects STDIN and STDOUT on your terminal to STDIN and STDOUT inside the first container in the Pod. Your shell prompt will change to indicate your shell is now connected to the container.

If you're running multi-container Pods, you'll need to pass the `--container` flag and give it the name of the container you want to create the exec session with. If you don't specify this flag, the command will execute against the first container in the Pod. You can see the ordering and names of containers in a Pod with the `kubectl describe pods <pod>` command. You can also inspect the Pod's YAML file.

Pod hostnames

Every container in a Pod inherits its hostname from the name of the Pod. This means all containers in a multi-container Pod get the same hostname.

The Pod you deployed is named `hello-pod` in the `pod.yml` file.

```
kind: Pod
apiVersion: v1
metadata:
  name: hello-pod      <<<=== This line
  labels:
  <Snip>
```

Run the following command from within the interactive exec session you already have to the Pod. The command is case-sensitive.

```
$ env | grep HOSTNAME
HOSTNAME=hello-pod
```

The hostname of the container is set to the Pod's name. With this in mind, you should always set Pod names as valid DNS names (a-z and 0-9 – the minus sign and the period sign).

Type `exit` to drop back to the terminal of your local machine.

Check Pod immutability

Pods are designed as immutable objects. You can use `kubectl edit` to try and update some Pod attributes.

Try editing any of the following,

- Pod name
- Container port
- Container name

Kubernetes will prevent you from changing them.

Multi-container Pod – init container

The following YAML defines a multi-container Pod with an init container and main app container.

```
apiVersion: v1
kind: Pod
metadata:
  name: initpod
  labels:
    app: initializer
spec:
  initContainers:
  - name: init-ctr
    image: busybox
    command: ['sh', '-c', 'until nslookup k8sbook; do echo waiting for k8sbook service;\
              sleep 1; done; echo Service found!']
  containers:
    - name: web-ctr
      image: nigelpoulton/web-app:1.0
      ports:
        - containerPort: 8080
```

The `spec.initContainers` block defines one or more containers that Kubernetes guarantees will run and complete before main app container starts.

This example has a single init container called "init-ctr" that loops until a Kubernetes Service object called "k8sbook" is up and present.

Deploy it with the following command and then run a `kubectl get pods` with the `--watch` flag.

```
$ kubectl apply -f initpod.yml
pod/initpod created

$ kubectl get pods --watch
NAME      READY   STATUS     RESTARTS   AGE
initpod   0/1     Init:0/1   0          5s
```

The `Init:0/1` status tells you that zero out of one init containers has completed successfully. The Pod will remain in this phase until a Service called "k8sbook" is created.

Create the Service and watch the Pod status change.

```
$ kubectl apply -f initsvc.yml
service/k8sbook created

$ kubectl get pods --watch
NAME      READY   STATUS     RESTARTS   AGE
initpod   0/1     Init:0/1   0          18s
initpod   0/1     PodInitializing   0          25s
initpod   1/1     Running           0          2m49s
```

As soon as the Service appears, the init container successfully completes, allowing the main application container to start.

Multi-container Pod – sidecar container

Sidecar containers are architecturally different to init containers. The sidecar is long-lived and runs alongside the main application container for the entire lifespan of the main container.

The following YAML file defines a Pod with two containers. Both mount the same volume and use it to share data.

The first container is called "ctr-web" and is the main app container. It serves a static web page based on content loaded from the shared volume. The second container is called "ctr-sync" and is the sidecar. It syncs content from a GitHub repo into the same shared volume. Net result, if the contents of the GitHub repo change, the "ctr-sync" sidecar container will copy the new content into the shared volume where the "ctr-web" container will notice and update the web page.

The mechanics of how it all works isn't really important here. What's important is seeing how two containers are defined in a single Pod YAML.

```
apiVersion: v1
kind: Pod
metadata:
  name: git-sync
  labels:
    app: sidecar
spec:
  containers:
  - name: ctr-web
    image: nginx
    volumeMounts:
    - name: html
      mountPath: /usr/share/nginx/
  - name: ctr-sync
    image: k8s.gcr.io/git-sync:v3.1.6
    volumeMounts:
    - name: html
      mountPath: /tmp/git
    env:
    - name: GIT_SYNC_REPO
      value: https://github.com/nigelpoulton/ps-sidecar.git
    - name: GIT_SYNC_BRANCH
      value: master
    - name: GIT_SYNC_DEPTH
      value: "1"
    - name: GIT_SYNC_DEST
      value: "html"
  volumes:
  - name: html
    emptyDir: {}
```

To see it in action, you'll need to fork the following GitHub repo (you'll need a GitHub account to be able to do this).

```
https://github.com/nigelpoulton/ps-sidecar
```

Update the `GIT_SYNC_REPO` value in the `sidecarpod.yml` and save your changes.

Deploy the application (Pod and Service).

```
$ kubectl apply -f sidecarpod.yml
```

You can now use a web browser to connect to the app via the Service.

It's currently displaying content from your forked repo. To see the sidecar container in action, make a change to the contents of your forked repo, wait a moment, and then refresh your browser tab viewing the app.

Feel free to run the `kubectl get pods` and `kubectl describe pod` commands to see how multi-container Pods appear in the outputs.

There's obviously a lot more to Pods than what we've covered. However, you've learned enough to get started.

Clean-up

Use `kubectl delete pods` and `kubectl delete svc` to delete any Pods and Services still running form this chapter. You'll need to provide the name of the Pods and Services you're deleting.

The following command deletes the `git-sync` Pod. It may take a few seconds for it to gracefully terminate.

```
$ kubectl delete pod git-sync
pod "git-sync" deleted
```

Chapter Summary

In this chapter, you learned the atomic unit of deployment in Kubernetes is the *Pod*. Each Pod has one or more containers and gets deployed to a single node in the cluster as an all-or-nothing *atomic operation*.

Pods are defined and deployed declaratively using YAML manifest files, and it's normal to deploy them via higher-level controllers such as Deployments and DaemonSets. If a Pod is not deployed via a controller it's called a *static Pod*.

You use `kubectl apply` to `POST` the YAML manifests to the API server, and Kubernetes picks a node to run the Pod on.

The `kubelet` daemon on the assigned worker node is responsible for pulling the strings to get the Pod started and then monitoring it and attempting local fixes.

If the node a static Pod is running on fails, the missing Pod doesn't get replaced by another one on another node.